Connect Suggests

Connect suggests. The owner approves once. Every year it needs them less.

SECTION 1

Summary

The problem. Fonecta Connect publishes each business's opening hours to more than 35 apps, including Google, Apple Maps and AI assistants, but owners forget to enter one-off holiday hours, so the apps keep showing normal hours and customers walk to a locked door. Asking the owner is not enough on its own: in Fonecta's 2025 pilot, 53% of suggestions got no reply.

What we built. Connect Suggests runs all five steps of Fonecta's brief end to end, on Fonecta's own test account at Yext, the service through which Fonecta Connect publishes listings. It finds the special days that each app will get wrong, predicts the real hours from the business's own past, and sends the owner one message per season in the chat they already use (WhatsApp, email or text message). The owner approves, changes or declines with no login. Connect then writes the change through Yext, reads it back and tells the owner where it shows.

The main innovation: “approve once, checked every year”. Next to *Approve*, the owner can tap *Approve + every year*. Fonecta's brief asked for this kind of standing permission; our addition is a safety check. Next season, Connect re-applies the owner's earlier answer by itself only if this year's prediction model agrees with it, meaning the model gives that answer at least a 50% chance. Otherwise it asks. Re-using last year's answers blindly gets open and closed wrong in 10.1% of automatic changes; with the check, 3.5%. This is how Connect reaches the owners who never reply.

61.67	86%	10.1% → 3.5%	36.8 vs 14.2
score on Fonecta's own scoring script, against 54.49 for Fonecta's baseline (its example solution)	of our suggestions are exactly right	automatic changes that mix up open and closed: blind re-use, then with our check	score for businesses with no history at all, against the baseline

The score counts points per 100 days that need a change, on Fonecta's points table: +1 for exactly right hours, −1 for mixing up open and closed (Section 6). All data is synthetic, from Fonecta's challenge dataset.

How to read this document

Each section starts in plain words. A shaded paragraph, *For the technical reader*, adds depth. Every number comes from Fonecta's own scoring script or from a script in our repository; the commands are in the Appendix. Nothing here contradicts the pitch deck; this document gives the how and the why behind it.

1 Summary
2 The problem and what the data told us
3 The five steps: the brief and what we built
4 The main innovation: Season Autopilot
5 The prediction engine, explained
6 Benchmarks
7 Why we beat the baseline
8 A brand-new business with zero information
9 Customer experience
10 Reliability: how wrong suggestions are prevented
11 Learning: the architecture
12 Extensibility
13 Technologies
14 Integration into Fonecta, and honest status
15 Appendix: reproduce, data sources, licences

SECTION 2

The problem and what the data told us

Weekly hours are usually right. Holiday hours are not: they change once a year, owners forget to enter them, and the apps keep showing normal hours. The customer finds a locked door.

Fonecta's challenge dataset holds 3,000 synthetic businesses in Finland, Sweden and Norway, with three years of history. For every business and every special day from 7 November 2026 to 6 January 2027 (41,755 business-days), a solution decides one of three actions: suggest hours, do nothing, or flag the business for a person. A *special day* is any day that may have unusual hours: public holidays, their eves, the days between them, and commercial days such as Black Friday. Before designing anything, we measured the problem.

"Copy last year" can only see 44% of the needed changes



Reply rate per suggestion in the 2025 pilot, by channel



Finding	The number	What it meant for our design
Closed doors are the main harm	63% of the needed changes are "shown open, actually closed": the app says open, the door is locked.	Switching a day between open and closed needs the strongest evidence.
Silence is the real bottleneck	In the 2025 pilot, 53% of suggestions got no reply (52% of owners never answered). Reply rate per suggestion: WhatsApp 65%, SMS 50%, email 40%, app notifications 32%.	Design for the owner who doesn't answer, not only for the one who does (Section 4).
Trust depends on the reason	Owners accepted 95% of suggestions based on their own history, against 81% of those based on similar businesses.	Lead with the owner's own history ("as you told us"); phrase similar-business suggestions as a question.
"Copy last year" sees under half	Of the needed changes, 44% have last season's entry; 16% exist only two seasons back; 36% show up only in the owner's other holidays.	Read two seasons back and the owner's other holidays.
Owner entries are accurate but biased	Entries match the truth 99% of the time, but owners entered 39% of special days, against 2% of normal days.	Train the models on the true hours, not on the entries (Section 7).
The provided peer file leaks future answers	The similar-business ("peer") file pools 2023–2025, so in a 2025 back-test (a replay of 2025 using only data known before it) it already contains the answers.	Recompute similar-business statistics ourselves, only from data known at the time.
Apps update at different speeds	Google about 2 minutes, Apple Maps about 2 days, AI assistants about 7 days.	Every message has an "answer by" date that gives the slowest app time.

For the technical reader. Every number is reproducible in `notebooks/01_data_and_models.ipynb`. A *needed change* is a business-day whose shown hours differ from the true hours: 5,376 of the 11,100 development ("dev") rows, plus 22 rows of businesses closed for good. Dev rows are the part of the dataset whose true hours are given. Update speeds come from `listings.json`; the leaking file is `peer_benchmarks.csv`. Two more findings shaped the engine: habits repeat (92% of businesses keep exactly the same hours on a holiday one season later, 87% after two seasons, 81% after three), and holidays are linked (P(closed on New Year's Day | closed on Christmas Day) = 0.89 against 0.36).

SECTION 3

What the brief asked for, and what we built: the five steps

Fonecta asked for a working demo of the whole chain and said "a complete chain beats a perfect single step". All five steps run end to end on Fonecta's Yext sandbox, a test account on the same system Fonecta Connect runs on. A full run takes about 28 seconds.

1 Detect

The brief: which data says the hours need to change?

For every business and special day, Connect rebuilds exactly what Google and the other apps will show if nothing is done: the weekly hours, any holiday hours already entered, and the rules for closures and seasonal reopening. It compares that with the engine's prediction of the real hours (step 2). Where the two differ, the day needs a change.

What we added beyond the brief: an exact rebuild that matches Fonecta's answer key on all 22,218 business-days, where Fonecta's sample code gets 86 days wrong in 2026; and a live special-day calendar that reproduces all 244 calendar rows of the dataset.

2 Suggest

The brief: which hours, and why exactly those?

The prediction engine (Section 5) estimates the real hours from the business's own past first (last season, two seasons back and its other holidays), then from similar businesses and the shopping centre it sits in. Every suggestion carries its reason and a confidence, with references to the data behind it. A decision rule sends only the suggestions worth sending. A business that may have closed for good goes to a person (a “flag”).

What we added beyond the brief: honest confidence (“80% sure” is right about 8 times in 10), a decision rule that maximises points on Fonecta's own scoring table, and a safety margin before switching a day between open and closed. 86% of suggestions are exactly right.

3 Contact

The brief: a message (email, SMS, WhatsApp…) with the suggestion and a clear reason.

One season card per business, in Finnish, Swedish, Norwegian or English, on the one channel the owner chose. It lists the days and hours with the reason (“as you told us”, “same as last year”), says what Google will show if the owner does nothing, and gives an “answer by” date. It keeps to each channel's limits (a WhatsApp message has at most 1,024 characters, a button at most 20), and the first message says it comes from an automated assistant. When the listing is already right, Connect stays silent.

What we added beyond the brief: an answer-by date set by the slowest app; one smart question per card, chosen by its value; and a test that checks every date and time in the message against the suggestion.

4 Decide

The brief: approve, edit or decline, with no login.

The owner answers inside the message: WhatsApp buttons, SMS digits, a signed one-tap link to a card page, or a reply in their own words, such as “kiinni tapaninpäivänä” (Finnish for “closed on Boxing Day”). Claude, the AI model we use for language, reads free text and reads it back to the owner before anything is saved. *Approve + every year* grants standing permission (Section 4).

What we added beyond the brief: opening a link never approves anything, so email link scanners cannot approve by accident; links are signed, expiring and single-use; a guessed switch between open and closed needs an explicit tick; and a rule-based reader serves as a fallback for the AI.

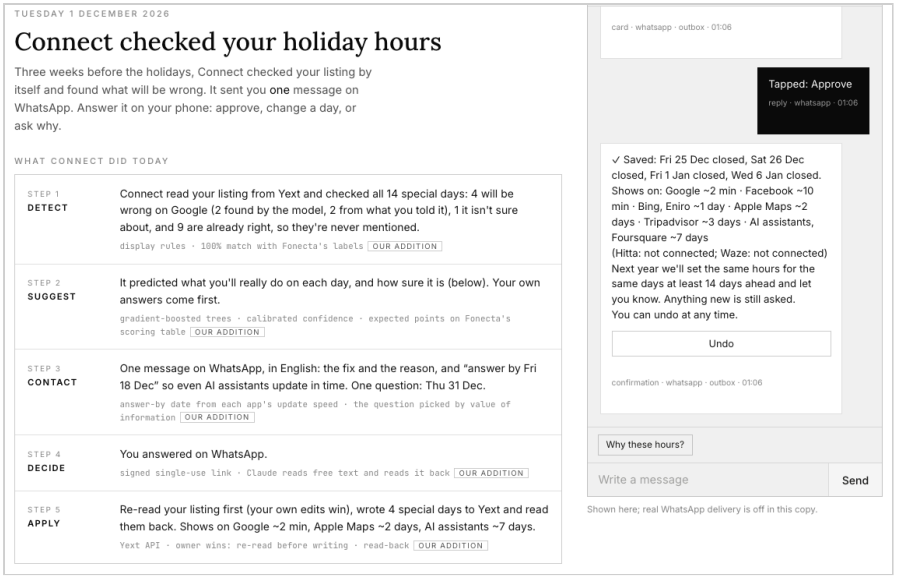
5 Apply and confirm

The brief: write the change to Connect's backend (the Yext API) and tell the owner where it now shows.

An API (application programming interface) is the way one program reads or writes another's data. Connect writes only the holiday hours through the Yext API, so weekly hours are never touched. It always sends the full list of future dates, because Yext deletes any date left out, and it drops past dates. It reads the hours back, and only then confirms, per app: “Google ~2 min · Apple Maps ~2 days · AI assistants ~7 days · Facebook: not connected”.

What we added beyond the brief: the listing is re-read just before writing, so if the owner changed a date themselves in the meantime, their change wins; and every suggestion, message, decision and write goes into a tamper-evident audit log.

For the technical reader. Detect is `cs/display.py`; suggest is `cs/engine/` and `cs/suggest/`, with the actions `SUGGEST`, `NO_ACTION` and `FLAG`; contact is `cs/compose.py` and `cs/channels/`; decide is `cs/links.py` (tokens signed with a secret key, HMAC, so a changed link is rejected; an HTTP GET never decides) and `cs/decide_flow.py`; apply is `cs/yext.py` and `cs/confirm.py`. Measured on the sandbox: a sent `holidayHours` list replaces the stored one, a left-out key keeps it, an empty list clears it, and past dates are silently dropped. No custom fields are added to Yext, as Fonecta advised; our own records live in MongoDB.



The demo app after the owner tapped Approve on WhatsApp: Connect wrote the hours, and the listing on Google is updated. Left: what Connect did at each step (demo view). Right: the owner's chat, which is all a real owner sees.

SECTION 4 · THE MAIN INNOVATION

"Approve once, checked every year" (Season Autopilot)

More than half of suggestions get no reply, so a loop that waits for approval never reaches the silent half. Season Autopilot lets an owner answer once and be looked after every year, without letting an old answer quietly go wrong.

> **What was in the brief, and what is ours.** Standing permission was in Fonecta's brief ("Learning: … and standing permission"). The model-agreement check is our addition.

1 One card, two equal buttons

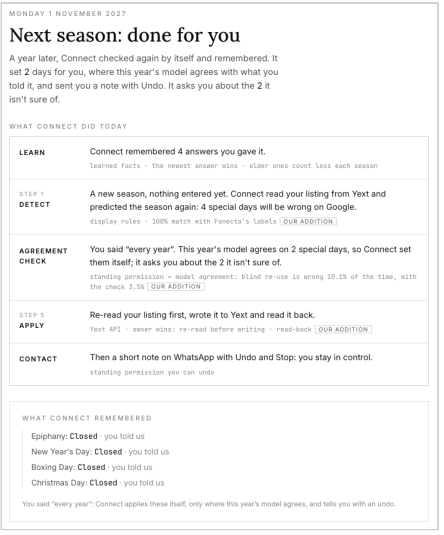
Approve, or *Approve + every year*. One sentence explains the scope: "every year" means the same days with the same hours; anything new or different is still asked; the owner can undo at any time.

2 The model-agreement check

Next season, Connect re-applies the owner's earlier answer for a day only if this year's prediction model gives that answer at least a 50% chance. Otherwise it asks about that day in the normal season card.

3 Applied early, with Undo and Stop

Agreed days are written at least 14 days ahead, and the owner gets a short note in their chat with Undo and Stop. Nothing is applied for a business that looks closed for good.



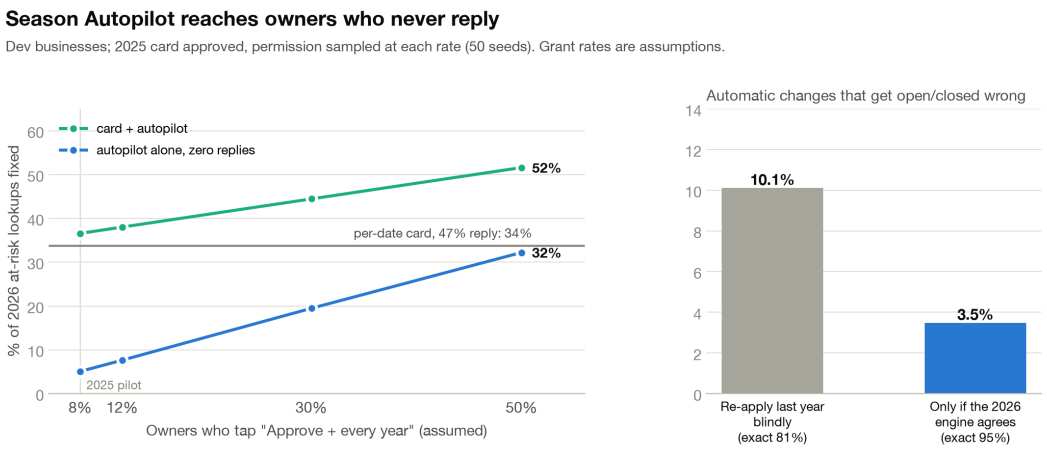
The demo one year later: Connect set 2 days itself, where this year's model agrees with the owner, and asks about the 2 it isn't sure of.

Automatic changes next season	Blind re-use	With the check
Exactly right	81%	**95%**
Open and closed mixed up	10.1%	**3.5%**

How much it reaches

An *at-risk lookup* is a customer search made on a day when the listing would be wrong. If 12% of owners say "every year", the season card plus Autopilot fixes 38.0% of next season's at-risk lookups, against 33.8% for the card alone. At 30% of owners it is 44.4%, and at 50% it is 51.6%. The check gives up about a third of the automatic changes' reach and in return cuts their error rate about three times.

An assumption, not a measurement: the share of owners who will tap “every year”. 12% is the fixed rate in Fonecta's owner simulator; the 2025 pilot's rate was 8% (57 of 761 owners), which gives 36.6%. Only a live test with real owners can measure it, so an A/B test of the button (showing two versions to comparable groups of owners and comparing the results) is the first thing we would run in production.



Left: share of next season's at-risk lookups fixed, by the assumed share of owners who tap “every year” (green: card plus Autopilot; blue: Autopilot alone with zero replies; grey line: the card alone). Right: automatic changes that get open and closed wrong, blind (10.1%) and with the agreement check (3.5%).

For the technical reader. `scripts/eval_autopilot.py`, on dev businesses, 50 random seeds per rate. The 2025 card is approved (an approving owner fixes what was wrong, so the learned facts are the true 2025 hours); a share of owners grants permission; in 2026 their dates are applied from the newest learned fact (older facts weigh 0.95 per season). Dates closer than 14 days, such as Father's Day on 8 November, are applied at once with a one-tap undo. A lookup counts as fixed when the hours are exact or within 30 minutes. The simulator's hidden owner profiles are never read. Each extra 10% of owners who grant permission adds about 3.5 points of coverage with no reply needed. The rule lives in `cs/autopilot.py` and is unit-tested for scope and agreement.

SECTION 5

The prediction engine, explained

For each business and special day, the engine answers two questions: will the business be closed, open as normal, or open with special hours, and if special, which hours? Then it decides whether a suggestion is worth sending.

It is built from established methods, chosen because they can be measured and explained; we don't claim new machine-learning research. What is new is how they are combined for this problem. Machine learning here simply means models that learn patterns from past examples.

Time honesty

A prediction made on 6 November may only use what was known on 6 November (the *snapshot*). Fonecta's data holds later information: 97% of the 2025-season entries were made after 6 November 2025, the provided peer file pools 2023–2025, and the business records describe the 2026 state. So every feature (an input the models learn from) passes through one time filter, and a unit test proves that the features don't change when everything after the snapshot is deleted.

The models

Model	What it does, in plain words	Measured
M1 · Habit types	Sorts businesses into 10 typical holiday habits, such as “closed on every church holiday, eves included” (25% of businesses) or “always regular” (5.5%). Found with an EM mixture (expectation–maximisation), a standard method for finding hidden groups.	K = 10 types, chosen on held-out data. Adds no measurable points; kept to explain habits in words and to model how an owner would answer the question.

Model	What it does, in plain words	Measured
M2 · State model	Predicts whether a day is closed, regular or special, with gradient-boosted trees (scikit-learn's HistGradientBoostingClassifier): many small decision trees, each correcting the mistakes of the ones before.	Closure AUC 0.970: 97 times in 100, it ranks a truly closed day above a day that is open.
M3 · Hours ranker	Lists candidate hours (the owner's own past, two seasons back, linked holidays, the shopping centre, similar businesses, map users' edits, shifted regular hours) and ranks them.	The true hours are among its candidates for 95% of needed changes; on special days its first choice is exact for 82% of those.
Isotonic calibration	Adjusts the models' confidence so that “80% sure” is right about 8 times in 10.	Calibration error 0.006 → 0.001.
M4 · Decision rule	For each day, picks the action (suggest, do nothing or flag) worth the most expected points on the scorer's exact table. Switching a day between open and closed needs an extra margin: 0.3 when the owner's own history supports the switch, 0.5 otherwise.	Margins chosen on other businesses; switches backed by the owner's own past are right about 95% of the time.
M5 · Closed-for-good flags	Rules: reviews saying the business has closed (in four languages) and “closed” edits from map users send the business to a person. Analytics (search and view counts) alone never count.	1 of 2 hidden closures caught, zero false alarms.
M6 · Weekly-hours drift	Logistic regression, a simple and explainable model, spots when a business's normal weekly hours have changed, mostly from reviews that state the new hours.	Precision 1.00: all 12 proposed weeks were right.

The one question

When the engine is unsure about a day, a guess may lock a door and silence may miss a fix. So a card may carry one question, about the day where an answer is worth the most. That worth is the *value of information*: how many expected points the owner's answer would add, given what the model already knows. Asking about that day gains +0.50 points per business, against +0.09 for a random date. The app asks about the best date not already on the card, so a question never repeats a suggestion. Honestly: the gain comes from choosing the date; the answer barely moves the other days.

Honest negatives

Linked holidays and habit types add no measurable points to the score. Peers, reviews, map users' edits and events add nothing on top of the business's own history. Bigger models only suggested more, with more harm. We report these because they tell Fonecta where the value really is: in each business's own past.

For the technical reader. Business-grouped cross-validation, 5 folds: every dev business is predicted by models that never saw it, and the calibration and the margins for each fold are chosen on the other folds (target: harmful ≤ 0.95 × 224). No 2026 label is used for training. Ablations, each re-running the whole cross-validation without one signal group (`scripts/eval_ablations.py`); differences within ±0.5 points are the noise of choosing the margins:

Variant	Score	Harmful	Variant	Score	Harmful
Full engine	61.67	219	Without habit types and linked holidays	62.00	222
Without any own history	33.43	198	Without peers	61.90	216
Without two seasons back	55.75	194	Without user edits and reviews	62.09	214
Without anything from other holidays	60.37	206	Without events and school breaks	62.05	215

The one question was measured on 150 dev businesses with at least one needed change (`scripts/eval_question.py`). Drift (M6) was evaluated with 5 × 5-fold cross-validation over 802 dev businesses: recall 0.55 (12 of 22 outdated listings found), and all 40 changed weekdays in the 12 proposals right.

SECTION 6

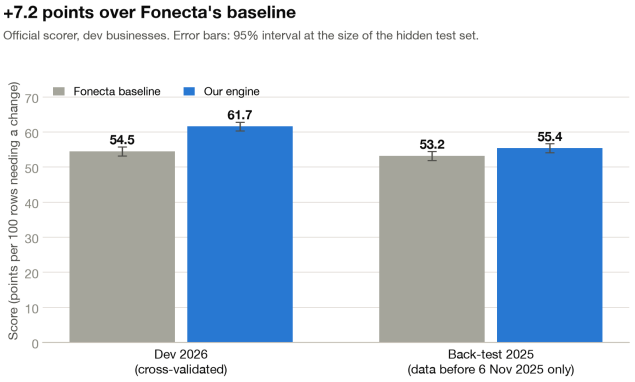
Benchmarks

All scores come from Fonecta's own scoring script (`evaluate.py`). The baseline is Fonecta's own example solution.

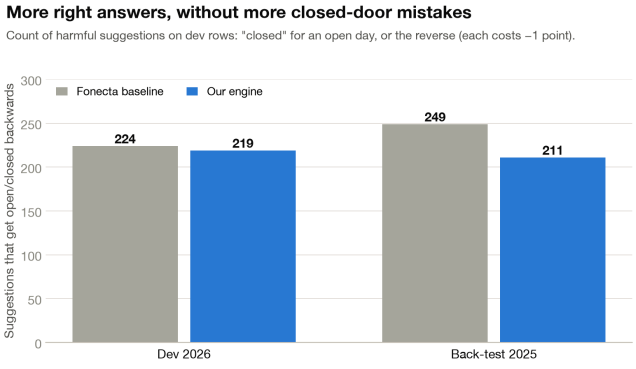
How the score works. For each business-day that needs a change: +1 for exactly right hours, +0.6 for the right open or closed state with both ends within 30 minutes, +0.2 for the right state only, +0.05 for a flag, and −1 for mixing up open and closed. On a day that is already right, any suggestion costs points (−0.1 to −1). The score is 100 × points ÷ the number of days that need a change or a flag.

	Score	Harmful	Exact precision	Recall	Search views protected	Closed-door views prevented
Our engine, dev 2026 (grouped cross-validation)	**61.67**	**219**	**0.859**	**0.670**	**65.0%**	**72.9%**
Fonecta baseline, dev 2026	54.49	224	0.901	0.590	57.2%	69.2%
Our engine, 2025 back-test	**55.42**	**211**	**0.866**	**0.597**	**56.3%**	**66.2%**
Fonecta baseline, 2025 back-test	53.23	249	0.899	0.578	55.1%	67.7%

Harmful: suggestions that mix up open and closed. **Exact precision:** the share of our suggestions with exactly the right hours. **Recall:** the share of days needing a change that we fix. **Search views protected:** the share of expected customer searches on wrong-hours days that our exact or near suggestions fix. **Closed-door views prevented:** the same, for days shown open that are actually closed. **Dev 2026:** Fonecta's development businesses, 2026 season. **Back-test:** replaying the 2025 season using only data from before 6 November 2025.



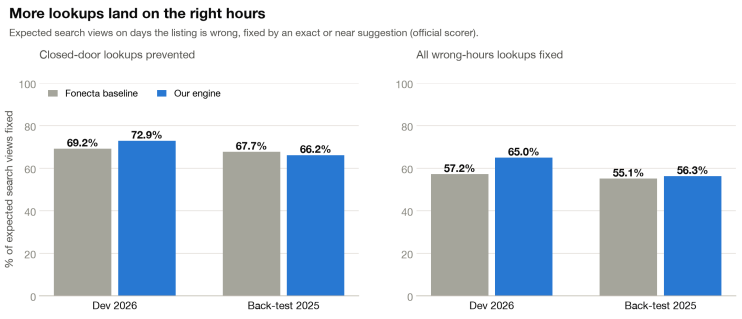
Score, with 95% ranges at the size of the hidden test set.



Harmful suggestions (open and closed mixed up; each costs one point).

How it was measured

Business-grouped cross-validation: the dev businesses are split into five groups, and each group is scored by models trained on the other four, so no business was scored by a model that saw it. Bootstrapping (re-sampling the data many times) gives a 95% confidence range of 60.4–62.9 for our score at the size of the hidden test set, and +6.3 to +8.1 for our lead of +7.2 points. Running Fonecta's `evaluate.py` on our submitted `predictions_2026.csv` reproduces 61.67 and 219 harmful exactly. The back-test gains less (+2.2), because in 2025 only one earlier winter of entries existed, and "two seasons back" is one of the strongest signals.



Expected search views on wrong days that our suggestions fix. On the back-test we prevent slightly fewer closed-door views (66.2% against 67.7%) while fixing more overall.

Honest caveats

- 219 harmful against the baseline's 224 is within noise: the range at test size is 200–239. On the back-test the gap is wider: 211 against 249.
- Our precision is 86% against the baseline's 90%. That is a deliberate trade: we make 433 more exact fixes.

```
.venv/bin/python dataset/FONECTA_CONNECT_SUGGESTS_2026/evaluation/evaluate.py --predictions predictions_2026.csv
```

SECTION 7

Why we beat the baseline

Five reasons, each measured, and one honest note on what doesn't add points.

1 **An exact rebuild of what every app shows**

A suggestion is only as good as knowing what the app shows today. Our rebuild matches Fonecta's answer key on 100% of 22,218 days; Fonecta's sample code gets 86 days wrong in 2026. It handles four edge cases where the provided code is wrong:

1. when a business that closed for good stops showing hours;
2. entries made on the snapshot day itself;
3. new weekly-hours versions that take effect on the snapshot day;
4. seasonal reopening, which restores the earlier hours.

22,218 of 22,218
days matched

2 **Reading two seasons back**

16% of needed changes only show up two seasons back, in years the owner didn't enter. Removing this signal alone drops the score from 61.67 to 55.75. It is the most valuable addition over “copy last year”.

+5.9 points
from this alone

3 **Training on the true hours, not on biased entries**

Owners enter 39% of special days but only 2% of normal days, so a model trained on entries learns a skewed picture. We train the open-or-closed model on the true hours of the dev businesses and use entries only as inputs and candidate hours.

39% vs 2%
special vs normal days entered

4 **Leak-free similar-business statistics**

The provided peer file pools 2023–2025, so in a back-test it already contains the answers. We recompute similar-business statistics only from entries made before the snapshot, in groups of at least 10 businesses.

No leak
proved by a test

5 **A calibrated decision rule with a safety margin**

Because the confidence is calibrated, the rule can weigh each action's expected points on Fonecta's table. It also needs an extra margin before switching a day between open and closed.

+7.2 points
over the baseline in total

Honest note: linked holidays and habit types add no measurable points, so we don't credit them for the score. The gradient-boosted trees already pick up the cross-holiday pattern from simple summaries of each business's entries.

SECTION 8

A brand-new business with zero information

Kahvila Kipinä (FI-D0017) is a café in Turku that joined on 12 October 2026. It has no holiday history and no earlier decisions, so anything Connect suggests must come from other businesses' data.

1 **Start from similar businesses**

The habit model's starting guess for businesses of the same type and size (the “segment prior”), plus statistics from groups of at least 10 similar businesses, computed without the future leak.

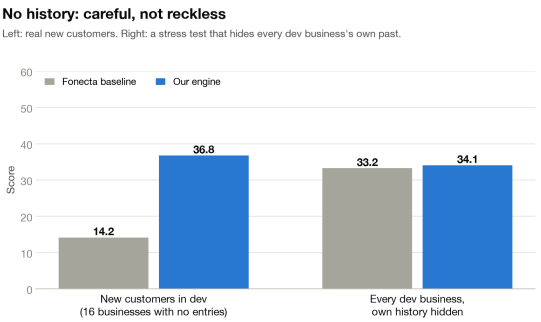
36.8 vs 14.2

score on the 16 dev businesses with truly no history (219 business-days), Connect against the baseline, with 2 harmful suggestions against 4

2. **Stay quiet unless near-certain**
   Only 1 of Kipinä's 14 special days gets a suggestion: New Year's Day closed, about 80% sure, based on similar cafés. The other 13 days are left alone.

3. **Spend the one question where it counts**
   The most valuable days to ask about are New Year's Day (+0.90 expected points) and Christmas Day (+0.87). New Year's Day is already on the card, so the question goes to Christmas Day.

4. **Let onboarding fill the gap**
   The welcome message asks how the business handles public holidays. Two taps answer it for every public holiday, so the first card can say “as you told us” instead of guessing.



Left: the 16 real new customers. Right: a stress test that hides every dev business's own past.

The gain comes mostly from not guessing where the baseline guesses. The stress test keeps us honest: with every dev business's own history hidden, Connect scores 34.1 and the baseline 33.2. Without a business's own past, similar businesses, the shopping centre and the segment prior recover only about a third of the points, for any method. One answer from the owner is worth more than any amount of data from similar businesses, which is why onboarding asks.

SECTION 9

Customer experience

The owner never installs an app, creates a password or fills in a form. Everything happens in the chat they chose when joining: WhatsApp, email or text message.

1. **Joining takes two taps.** The welcome message arrives on the chosen channel with one question, “On public holidays you're usually: Closed / Shorter hours / Open as normal”, and then asks whether that holds every year. The answers become Connect's memory. The first message says it comes from an automated assistant, and that replying STOP opts out.
2. **One message per season.** About three weeks before the holidays, Connect checks the listing by itself and sends one card for the whole season. If the listing is already right, the owner hears nothing.
3. **An answer-by date set by the slowest app.** Google updates in about 2 minutes, but AI assistants take about 7 days, so the card says, for example, “Please answer by Sat 19 Dec so AI assistants show it in time.”
4. **One smart question.** If the engine is unsure about a day, the card asks about the one date where an answer is worth the most.
5. **No login.** One tap on a button, or a reply in their own words. If the owner writes “we close at 14 on the 24th”, Claude reads it and Connect asks “Shall we save this? • Thu 24 Dec 07:30–14:00 [Yes] [No]” before anything is saved.
6. **“Closed today.”** For the unplanned locked door, such as illness or a burst pipe, the owner texts “closed today” (“kiinni tänään”) or taps a button. It is live on Google within minutes, ends by itself, and the reason is never stored.
7. **Questions answered with sources.** “Is the shopping centre open on 26.12?” or “What did I do last Easter?” are answered from the business's own data, with a source for every fact (Section 11).

Connect Suggests WHATSAPP

Connect Suggests · automated assistant
Hi Restaurang Fyren! Christmas and New Year hours – our suggestion:
• Sat 26 Dec closed (as you told us)
• Wed 6 Jan closed (as you told us)
If you do nothing, Google shows on these days: 11:00–23:00
Please answer by Sat 19 Dec so AI assistants show it in time.

Approve

Approve + every year

Edit

A season card, as text (from the demo).

8 **Undo and Stop.** Every change and every automatic note carries Undo; Stop ends the standing permission.

The web dashboard in our demo exists only to show what happens behind the scenes. A real owner does practically everything in their chosen chat: joining, approving, editing, declining, “closed today”, questions, Undo and Stop. The only other screen is the optional no-login card page that the message links to.

SECTION 10

Reliability: how wrong suggestions are prevented

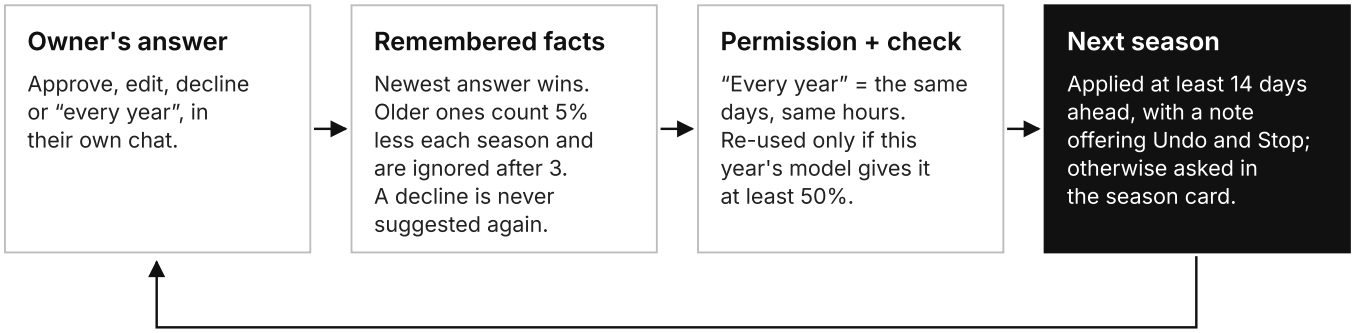
The owner always has the last word: nothing is written without their decision or their standing permission, every change can be undone, and the owner wins any conflict. Below that, wrong suggestions are stopped in layers.

Layer	Safeguard	Evidence
Before any message	An exact rebuild of what each app shows today	100% of 22,218 days
	A decision rule with safety margins for switching open and closed	219 harmful against 224
	Hard rules: never contradict the owner's own entry; no past dates; valid hours only; a “no” is never suggested again; opted-out owners are never messaged; no suggestions to listings closed for good	Unit-tested
	A business that may have closed for good goes to a person	1 of 2 caught, zero false alarms
When the owner decides	A guessed switch between open and closed needs an explicit tick	Built into the card
	A reply in the owner's own words is read back before saving	Rule-based fallback for the AI
	Links are signed, expiring and single-use; opening a link never approves anything, so email link scanners can't approve by accident; WhatsApp replies arrive signed by Meta	Tests for expiry, reuse, tampering, “opening never decides” and spoofed senders
When writing to Yext	The listing is re-read first, so the owner's own edits win	Tested on the sandbox
	Only holiday hours are written; weekly hours are never touched; the full future list is sent, past dates dropped; the hours are read back before the ✓ appears	Tested
	Every step goes into a tamper-evident audit log: each record is chained to the one before it	A “Verify” check for staff
When acting alone	Only within the owner's scoped permission: the same days with the same hours	Unit-tested scope
	The model-agreement check: re-used only if this year's model gives it at least 50%	10.1% → 3.5% wrong
	Applied at least 14 days ahead with a note offering Undo and Stop; nothing applied for a business that looks closed for good; “closed today” ends by itself	Tested, incl. auto-expiry
AI safety	Facts always come from data: Claude reads replies and polishes wording, under a check that no date or time drifts	A test checks every date and time against the suggestion
	Answers to questions cite a source for every fact; a checker rejects any date, time or number not in the facts	Tested with invented times
	Privacy: no names or contacts are sent to the AI; reasons for closing and review authors' names are never stored	“No reason stored” test
	Disclosure: the first contact says it is an automated assistant (EU AI Act, Article 50); everything also runs with the AI switched off	Setting `LLM_ENABLED=0`

SECTION 11

Learning: the architecture

Every answer makes next season better. Owners' answers become remembered facts, and “every year” becomes a scoped permission, protected by the agreement check.

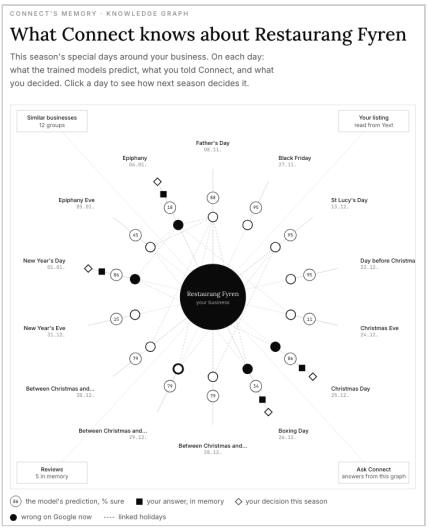


Each season's answers become the next season's facts.

The learning loop. The 5% decay follows the data: 92% of businesses keep the same holiday hours one season later, 87% after two, 81% after three. Declines are never repeated because, when an owner rejected a 2025 suggestion, the 2026 truth equalled the rejected hours only 7% of the time.

A memory graph for every business

A *graph* stores facts as points (nodes) joined by labelled links (edges). Each business has its own small graph in MongoDB, a database. The node types are BUSINESS, SPECIAL_DAY, ENTRY (hours entered), LEARNED_FACT (a remembered answer), DECISION, MALL (the shopping centre), PEER_GROUP (at least 10 similar businesses, never one individual), REVIEW and EVENT, linked by typed edges such as “entered”, “decided” and “supersedes”. Every node records where its fact came from.



The demo's view of one business's memory: each special day with the model's prediction, the owner's answer and this season's decision.

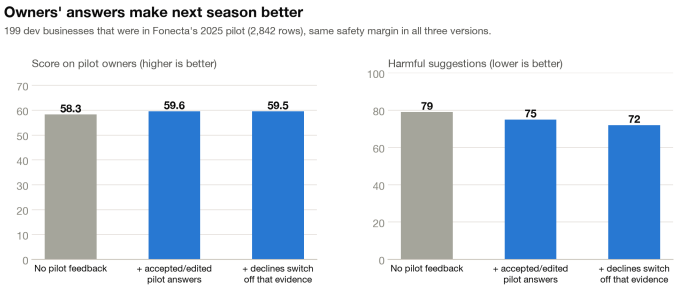
Graph RAG: answers with a source for every fact

RAG, retrieval-augmented generation, means the AI first retrieves facts and then writes an answer only from them. Our retrieval follows the graph's links from the days a question is about. It uses no embeddings (the numeric text fingerprints most RAG systems use for fuzzy search), so what is retrieved is exact and explainable. Claude answers only from the retrieved facts, with a source for every fact, and a checker rejects any date, time or number that is not in the facts. Asked “Is the shopping centre open on 26.12?”, Connect answers: “Yes, the shopping centre entered 11:00–17:00 for 26.12.2026 (Boxing Day) on 22.10.2026 [holiday_hours_history:FI-M000:2026-12-26] … Connect suggests 11:00–17:00 (88% confidence) [engine:FI-D0004:2026-12-26].”

Measured

On the 199 dev businesses that were in Fonecta's 2025 pilot (2,842 business-days), with the same safety margin throughout: the score rises from 58.3 without the pilot answers to 59.6 with them, and harmful suggestions fall from 79 to 75 with the accepted and edited answers, then to 72 once declines switch off that evidence.

> **Honest note:** the prediction models don't read the graph. It is the memory for questions and for next season's check. The learning effect on the score was measured offline, by re-running the engine with and without the pilot answers.



Score and harmful suggestions on the 2025 pilot owners.

For the technical reader. Learned facts and permissions: `cs/learn.py` (a newer answer supersedes; weight 0.95 per extra season; facts older than three seasons drop out unless reconfirmed; undo revokes a permission, an edit updates it). Graph: `cs/kg/build.py`, with the edge types ENTERED, DECIDED, LEARNED, SUPERSEDES, FOR_DAY, IN_MALL, SIMILAR_TO, MENTIONS, LINKED_HOLIDAY and NEAR_EVENT; every node carries `source` and `ref`, and only data stamped before the snapshot enters. Retrieval: `cs/kg/retrieve.py` maps holiday names and dates in Finnish, Swedish, Norwegian or English to days, then walks up to two hops. Answering and checking: `cs/kg/answer.py`.

SECTION 12

Extensibility

Every conversation is the same loop: detect → message → the owner decides → write to Yext. The loop already runs six kinds of conversation.

Built and working

Holiday hours
the season card

Weekly hours that changed
precision 1.00: all 12 proposed weeks right

Replies to reviews
drafted for the owner to approve, not posted

“Closed today”
on Google within minutes, ends by itself

Onboarding
two taps in the owner's chat

The Autopilot note
with Undo and Stop

A new use needs only three parts

A detector → **A message template** → **A Yext write**

Candidates (not built yet)

- Seasonal hours
- Event hours
- Sorting map users' suggested edits
- Phone and address changes
- Photos
- Attributes (listing details)
- Posts

SECTION 13

Technologies

Technology	What it does here	Why we chose it
scikit-learn HistGradientBoosting	Predicts whether a day is closed, regular or special, and ranks candidate hours	The strongest general choice for table-shaped data of this size; retrains in about 6 minutes on a laptop; deep models aren't needed for 3,000 businesses and are harder to explain
Isotonic calibration	Makes the confidence honest: “80% sure” is right about 8 times in 10	The decision rule weighs expected points, which only works with honest probabilities (error 0.006 → 0.001)
EM habit model	Sorts businesses into 10 holiday habits	Explains a business's habits in words, gives the starting guess for new businesses and models how an owner would answer
Logistic regression	Spots weekly hours that have changed	Simple and explainable; precision 1.00
Value of information	Chooses the one question per card	A principled measure of what an answer is worth: +0.50 points per business against +0.09 for a random date
Claude Opus 5	Reads owners' replies in their own words, polishes wording and writes answers to questions	Strong where language is the problem; it writes its analysis first and a strict, machine-readable result last; facts always come from data
Graph RAG on MongoDB	Each business's memory graph; answers questions from it with sources	Following typed links instead of embeddings keeps retrieval exact and every fact citable; MongoDB also holds the audit log

Technology	What it does here	Why we chose it
Yext Knowledge API	Reads and writes listings, holiday hours only	The interface Fonecta Connect already publishes through, so nothing new is needed in Fonecta's backend
WhatsApp Cloud API, signed webhook	Sends the cards and receives replies at our web address (the webhook)	Meta's official API; WhatsApp has the highest reply rate (65%); the signature proves a reply really came from Meta
Flask and HTMX	The no-login card page, the demo dashboard and the staff pages	A small Python web server, and a light library that updates parts of a page without reloading it
SHA-256 hash-chained audit log	Records every suggestion, message, decision and write	Each record stores a fingerprint (SHA-256 hash) of the one before, so any later change breaks the chain and is detected

SECTION 14

Integration into Fonecta, and honest status

The same Yext API as Fonecta Connect. Connect Suggests reads and writes through the same Yext API that Fonecta Connect runs on. We tested it on Fonecta's own Yext sandbox, on the record SinceAi04, renamed Kahvila Aurora, so what we detect is what we write. No custom fields are added to Yext; our own records live in MongoDB.

No new app for owners. It lives in the chat they already use: WhatsApp, email or text message.

Honest status

- Replies from a real phone come through Meta's webhook.
- SMS is shown on screen only (Finnish operators block unregistered senders).
- Review replies are drafted, not posted.
- All data is synthetic.
- The share of owners who choose “every year” is an assumption until tested with real owners.

Going live is mostly configuration

- ☐ A production Yext key
- ☐ Fonecta's WhatsApp number and message templates
- ☐ An SMS or email provider
- ☐ Owner contacts and consent from the customer records
- ☐ One public web address
- ☐ A nightly scheduled run

A production launch would also use Fonecta's own sender accounts (with an authorised SMS sender ID under the Finnish regulator Traficom's rules), the real hours log and customer records instead of dataset files, EU hosting, staff sign-in for the staff pages, and one A/B test of the “every year” button.

SECTION 15

Appendix

A. Reproduce the results

Python 3.11 and the challenge dataset (included as a zip in `dataset/`). The score engine needs no keys.

```
python3 -m venv .venv && .venv/bin/pip install -r requirements.txt     # exact versions: requirements.lock.txt
unzip -q dataset/FONECTA_CONNECT_SUGGESTS_2026.zip -d dataset/

# Score the submitted predictions with Fonecta's scorer: 61.67, 219 harmful
.venv/bin/python dataset/FONECTA_CONNECT_SUGGESTS_2026/evaluation/evaluate.py --predictions predictions_2026.csv

# Retrain the engine and the 2025 back-test (about 6 min, CPU only)
.venv/bin/python scripts/train.py --refresh --backtest
.venv/bin/python scripts/eval_ablations.py      # optional, about 20 min
.venv/bin/python scripts/eval_question.py       # the one question
.venv/bin/python scripts/eval_drift.py          # weekly-hours drift
.venv/bin/python scripts/eval_autopilot.py      # Season Autopilot

# The five steps in the terminal, against Fonecta's local Yext mock
.venv/bin/python scripts/run_mock.py                    # in a second terminal
.venv/bin/python scripts/demo.py FI-D0001 --target mock

# Tests: 461 offline tests in about 15 s (537 in all, with MongoDB)
.venv/bin/pytest -q -m "not mongo and not network and not sandbox"
```

B. The data sources that mattered most

Each was measured by removing it and re-running the full pipeline.

Signal	How we used it	Worth (measured)	Where Fonecta would get it in production
The business's own past entries for the same holiday	The strongest evidence for state and hours	Without any own history: 61.7 → 33.4	Connect's log of entered holiday hours (Yext `holidayHours` history)
The entry from two seasons back	Covers years the owner didn't enter	−5.9 points without it	The same log, kept for at least 3 seasons
The business's other holidays	Linked holidays	−1.3 points and +21 harmful without it	The same log
Owners' past decisions (2025 pilot)	Answers become history; declines are never repeated	58.3 → 59.6; harmful 79 → 72	Connect's suggestion and decision log
What Google shows now	The starting point of every decision	100% match	The Yext Entities API

C. Licences

- **Data:** "Connect Suggests simulator, Since AI ry 2026", licensed CC BY 4.0. The dataset's code (mock API, `hours.py`, example client, scorer) is MIT-licensed.
- **Fonts:** Lora and Inter, under the SIL Open Font License 1.1.
- **Software:** permissive open-source licences only (BSD, MIT, Apache 2.0 and similar); no GPL or AGPL components. The full list is in README section 13.